BÀI TẬP THỰC HÀNH: XÂY DỰNG AI PHÂN BIỆT CHÓ VÀ MÈO (DOGS VS. CATS)

I. ĐỀ BÀI (TÌNH HUỐNG THỰC TẾ)

Bối cảnh: Bạn đang làm việc tình nguyện cho một trạm cứu hộ động vật. Trạm nhận được hàng nghìn bức ảnh động vật đi lạc mỗi ngày từ camera đường phố và người dân gửi đến. Việc phân loại thủ công xem đó là chó hay mèo tốn quá nhiều thời gian.

Nhiệm vụ của bạn: Xây dựng một mô hình CNN có khả năng tự động đọc ảnh từ thư mục, thay đổi kích thước ảnh (resize) và phân biệt chính xác chó và mèo.

Kiến thức trọng tâm mới:

1. Đọc dữ liệu ảnh trực tiếp từ thư mục (Folders) bằng ImageDataGenerator.
2. Bài toán Phân loại Nhị phân (Binary): Đầu ra chỉ cần **1 nơ-ron** với hàm kích hoạt **Sigmoid**.
3. Hàm mất mát chuyển từ Categorical sang **Binary Cross-Entropy**.

Mục tiêu KPI:

- Xây dựng mạng CNN cơ bản (Train from scratch): Đạt độ chính xác Validation > **75%**.

II. HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC (NOTEBOOK)

Bạn hãy chạy tuần tự các ô code dưới đây trên Google Colab.

[Code Cell 1: Tải dữ liệu thực tế]

Python
```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import os
import urllib.request
import zipfile

# 1. Tải bộ dữ liệu Cats & Dogs (Phiên bản rút gọn của Google - 3000 ảnh)
url = 'https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip'
file_name = 'cats_and_dogs.zip'
urllib.request.urlretrieve(url, file_name)
```

```
# 2. Giải nén dữ liệu
with zipfile.ZipFile(file_name, 'r') as zip_ref:
    zip_ref.extractall()

# 3. Cài đặt đường dẫn tới các thư mục
base_dir = 'cats_and_dogs_filtered'
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')

print("Đã tải và giải nén dữ liệu thành công!")
print(f"Thư mục Train chứa: {os.listdir(train_dir)}")
```

[Code Cell 2: Đọc ảnh từ thư mục & Tăng cường dữ liệu]

Trong thực tế, ảnh có kích thước rất lộn xộn (ví dụ: 800×600, 1024×1024). Chúng ta phải dùng flow_from_directory để tự động cắt/ép tất cả ảnh về một chuẩn chung, ví dụ 150×150 pixel.

Python

```
# Cài đặt kích thước chuẩn cho toàn bộ ảnh
IMG_WIDTH = 150
IMG_HEIGHT = 150
BATCH_SIZE = 32

# Bộ đọc dữ liệu Train (Có áp dụng Data Augmentation để chống Overfitting)
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Chuẩn hóa pixel về [0, 1]
    rotation_range=40,       # Xoay ngẫu nhiên
    width_shift_range=0.2,   # Dịch chuyển ngang
    height_shift_range=0.2,  # Dịch chuyển dọc
    shear_range=0.2,         # Làm méo ảnh
    zoom_range=0.2,          # Phóng to/thu nhỏ
    horizontal_flip=True     # Lật ngang
)

# Bộ đọc dữ liệu Validation (Tuyệt đối KHÔNG làm méo/xoay dữ liệu kiểm tra)
test_datagen = ImageDataGenerator(rescale=1./255)

print("Đang đọc dữ liệu Train:")
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary' # Phân loại 2 lớp (Chó/Mèo)
)

print("Đang đọc dữ liệu Validation:")
validation_generator = test_datagen.flow_from_directory(
    validation_dir,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)
```

[Code Cell 3: Thiết kế mạng CNN cho Phân loại nhị phân]

Điểm khác biệt lớn nhất: Lớp Dense cuối cùng chỉ có **1 nơ-ron** và dùng hàm **Sigmoid** (trả về xác suất từ 0 đến 1. Gần 0 là Mèo, gần 1 là Chó).

Python
```
model = models.Sequential()

# Block 1
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)))
model.add(layers.MaxPooling2D((2, 2)))

# Block 2
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Block 3
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Block 4
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Khối phân loại
model.add(layers.Flatten())
model.add(layers.Dropout(0.5)) # Chống học vẹt
model.add(layers.Dense(512, activation='relu'))

# OUTPUT LAYER: 1 Nơ-ron, hàm Sigmoid
model.add(layers.Dense(1, activation='sigmoid'))

model.summary()
```
[Code Cell 4: Biên dịch và Huấn luyện]

Vì là bài toán nhị phân (Binary), chúng ta bắt buộc phải đổi hàm Loss thành `binary_crossentropy`.

Python
```
from tensorflow.keras.optimizers import Adam

model.compile(loss='binary_crossentropy',
              optimizer=Adam(learning_rate=1e-4), # Tốc độ học nhỏ để mạng học từ từ
              metrics=['accuracy'])

print("Bắt đầu huấn luyện... (Có thể mất 5-10 phút)")
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // BATCH_SIZE, # Số bước mỗi vòng lặp
    epochs=30,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // BATCH_SIZE
)
```
[Code Cell 5: Vẽ biểu đồ đánh giá]

Python
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(len(acc))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs, acc, 'b', label='Train Accuracy')
plt.plot(epochs, val_acc, 'r', label='Validation Accuracy')
plt.title('Độ chính xác (Accuracy)')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epochs, loss, 'b', label='Train Loss')
plt.plot(epochs, val_loss, 'r', label='Validation Loss')
plt.title('Độ mất mát (Loss)')
plt.legend()
plt.show()
```

[Code Cell 6: Thử nghiệm thực tế với ảnh từ máy tính]

Đã đến lúc dùng thử AI của bạn. Hãy tải một tấm ảnh chó hoặc mèo bất kỳ trên mạng về máy tính và upload vào đây!

Python
```
import numpy as np
from google.colab import files
from tensorflow.keras.preprocessing import image

print("Bấm nút để tải ảnh Chó/Mèo từ máy tính lên:")
uploaded = files.upload()

for fn in uploaded.keys():
    # Load ảnh và tự động ép về chuẩn 150x150
    path = '/content/' + fn
    img = image.load_img(path, target_size=(150, 150))

    # Chuyển ảnh thành ma trận và chuẩn hóa
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = x / 255.0

    # Đưa vào mạng dự đoán
    prediction = model.predict(x)

    # Vẽ ảnh và in kết quả
    plt.imshow(img)
    plt.axis('off')

    # Vì dùng Sigmoid: Xác suất > 0.5 là Chó (class 1), ngược lại là Mèo (class 0)
    if prediction[0][0] > 0.5:
        plt.title(f"Dự đoán: ĐÂY LÀ CHÓ (Độ tự tin: {prediction[0][0]*100:.1f}%)", color='blue')
```

```
    else:
        # Nếu nhỏ hơn 0.5, độ tự tin là con số ngược lại
        confidence = (1 - prediction[0][0]) * 100
        plt.title(f"Dự đoán: ĐÂY LÀ MÈO (Độ tự tin: {confidence:.1f}%)", color='green')
    plt.show()
```

III. THỬ THÁCH NÂNG CAO

Chủ đề: Sức mạnh của Transfer Learning (Học chuyển giao)

Với mô hình tự xây ở trên, độ chính xác thường chỉ đạt khoảng **75-80%** (vì chúng ta chỉ có 2000 ảnh để train).

Yêu cầu: Hãy thay thế các khối Convolutional (Block 1 đến Block 4) bằng mạng **VGG16** đã được huấn luyện sẵn của Keras. Bạn chỉ cần giữ lại lớp Flatten và Dense ở cuối.

Nếu làm đúng, độ chính xác Validation của bạn sẽ vọt lên trên **90%** chỉ sau 5-10 vòng lặp!

Gợi ý code chèn mạng VGG16:

Python
```
from tensorflow.keras.applications import VGG16

# Tải phần thân (Backbone) của VGG16
conv_base = VGG16(weights='imagenet',
                  include_top=False,
                  input_shape=(150, 150, 3))

# Đóng băng các trọng số để không bị phá hỏng lúc huấn luyện
conv_base.trainable = False

# Ghép thân VGG16 với phần đầu (Head) của bạn
model = models.Sequential()
model.add(conv_base)
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Lưu ý:

- Ở bài này, **Lỗi kích thước ảnh (Shape Error)** là lỗi sinh viên gặp nhiều nhất. Chú ý sử dụng `ImageDataGenerator` là "vị cứu tinh" giúp tự động bóp méo/cắt xén ảnh gốc về đúng `(150, 150)` trước khi đưa vào mạng.
- Sự khác biệt giữa `Softmax` (tổng xác suất các lớp = 1) và `Sigmoid` (xác suất độc lập từ 0-1) là câu hỏi phỏng vấn vị trí AI Engineer rất phổ biến. Lưu ý kỹ mục này ở Bước 3.